Add the following dependency

```
implementation 'com.google.code.gson:gson:2.8.5'
```

create the following pojo

```
public class Employee {
    private String firstName;
    private int age;
    private String mail;

    public Employee(String firstName, int age, String mail) {
        this.firstName = firstName;
        this.age = age;
        this.mail = mail;
    }
}
```

Add the following code –

```
Gson gson = new Gson();

Employee employee = new Employee("John", 30, "john@gmail.com");
String json = gson.toJson(employee);
```

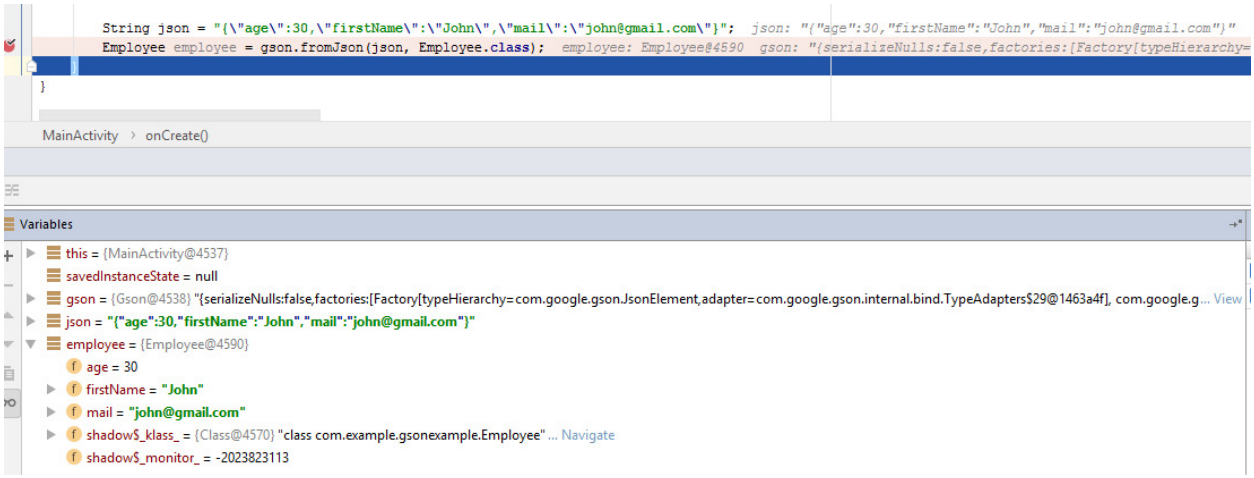
On executing this code it creates a json string as follows -

```
Json = {
  "age": 30,
  "firstName": "John",
  "mail": "john@gmail.com"
}
```

Similarly we can convert json into the object of employee class –

```
    String json = "{\"age\":30,\"firstName\":\"John\",\"mail\":\"john@gmail.com\"}";
    Employee employee = gson.fromJson(json, Employee.class);
```

The output can be seen as –



This could have possible because the member variable names of Employee class are same as that of json attribute names.

Let us take another case where member variable names are different from json attributes.

```
public class Employee {
    private String mFirstName;
    private int mAge;
    private String mMail;

    public Employee(String firstName, int age, String mail) {
        this.mFirstName = firstName;
        this.mAge = age;
        this.mMail = mail;
    }
}
```

in this case we have to annotate the member variables with

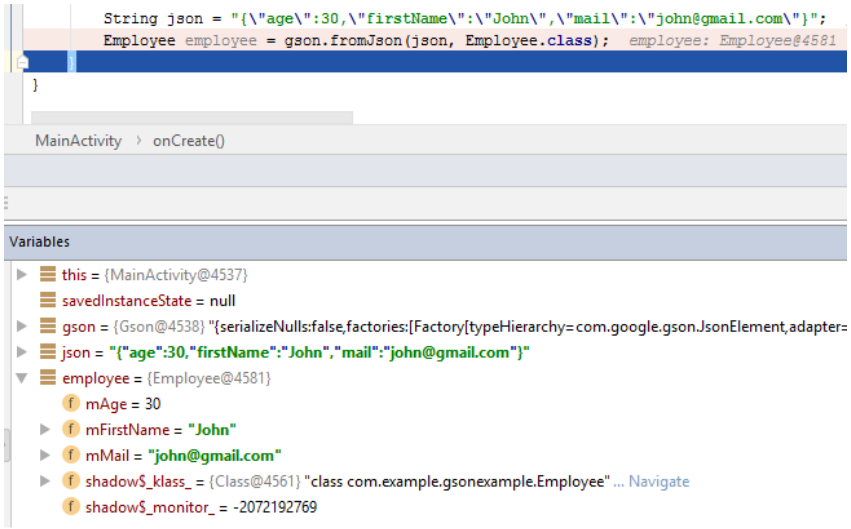
@SerializedName("json-attribute")

```
import com.google.gson.annotations.SerializedName;

public class Employee {

    @SerializedName("firstName")
    private String mFirstName;

    @SerializedName("age")
    private int mAge;

    @SerializedName("mail")
    private String mMail;

    public Employee(String fName, int age, String mail) {
        this.mFirstName = fName;
        this.mAge = age;
        this.mMail = mail;
    }
}
```

Run the code

```
String json = "{\"age\":30,\"firstName\":\"John\",\"mail\":\"john@gmail.com\"}";
Employee employee = gson.fromJson(json, Employee.class);
```

again. This gives the following output –



Nested JSON Objects

Now create an another class –

```
import com.google.gson.annotations.SerializedName;

public class Address{
    @SerializedName("country")
    private String mCountry;
    @SerializedName("city")
    private String mCity;

    public Address(String country, String city){
        mCountry = country;
        mCity = city;
    }
}
```

Now add this address object in Employee class –

```
import com.google.gson.annotations.SerializedName;

public class Employee {

    @SerializedName("firstName")
    private String mFirstName;

    @SerializedName("age")
    private int mAge;

    @SerializedName("mail")
    private String mMail;

    @SerializedName("address")
    private Address mAddress;

    public Employee(String fName, int age, String mail, Address address) {
        this.mFirstName = fName;
        this.mAge = age;
        this.mMail = mail;
        this.mAddress = address;
    }
}
```

Modify the code as follows –

```
Gson gson = new Gson();

Address address = new Address("India", "New Delhi");

Employee employee = new Employee("John", 30, "john@gmail.com", address);
String json = gson.toJson(employee);
```

this gives the following json –

```
{
  "address": {
    "city": "New Delhi",
    "country": "India"
  },
  "age": 30,
  "firstName": "John",
  "mail": "john@gmail.com"
}
```

Similarly we can convert a nested json into object of Employee class as follows –

```
Gson gson = new Gson();

String json = "{\n" +
        "  \"address\": {\n" +
        "    \"city\": \"New Delhi\",\n" +
        "    \"country\": \"India\"\n" +
        "  },\n" +
        "  \"age\": 30,\n" +
        "  \"firstName\": \"John\",\n" +
        "  \"mail\": \"john@gmail.com\"\n" +
        "}";

Employee employee = gson.fromJson(json, Employee.class);
```

As shown in below screenshot the json converts into object of Employee class without specifying the Address.class as it is enclosed in Employee class.

```
        String json = "{\n" +
                "  \"address\": {\n" +
                "    \"city\": \"New Delhi\",\n" +
                "    \"country\": \"India\"\n" +
                "  },\n" +
                "  \"age\": 30,\n" +
                "  \"firstName\": \"John\",\n" +
                "  \"mail\": \"john@gmail.com\"\n" +
                "}";
        Employee employee = gson.fromJson(json, Employee.class);
    }
}
```

```
Variables
▶ this = {MainActivity@4539}
  savedInstanceState = null
▶ gson = {Gson@4540} "{serializeNulls:false,factories:[Factory[typeHierarchy=com.google.gson.JsonElement,adapter=com.g
▶ address = {Address@4541}
▼ employee = {Employee@4542}
  ▼ mAddress = {Address@4541}
    ▶ mCity = "New Delhi"
    ▶ mCountry = "India"
    ▶ shadow$_klass_ = {Class@4536} "class com.example.gsonexample.Address" ... Navigate
      shadow$_monitor_ = -1912798935
  mAge = 30
  ▶ mFirstName = "John"
  ▶ mMail = "john@gmail.com"
  ▶ shadow$_klass_ = {Class@4537} "class com.example.gsonexample.Employee" ... Navigate
    shadow$_monitor_ = -1983238994
▶ json = "{"address":{"city":"New Delhi","country":"India"},"age":30,"firstName":"John","mail":"john@gmail.com"}"
```

Gson handling ArrayLists –

```
import com.google.gson.annotations.SerializedName;

public class FamilyMember {
   @SerializedName("role")
   private String mRole;
   @SerializedName("age")
   private int mAge;

   public FamilyMember(String role, int age){
      mRole = role;
      mAge = age;
   }
}
```

Modify the Employee class as follows –

```
import com.google.gson.annotations.SerializedName;
import java.util.List;

public class Employee {

    @SerializedName("firstName")
    private String mFirstName;

    @SerializedName("age")
    private int mAge;

    @SerializedName("mail")
    private String mMail;

    @SerializedName("address")
    private Address mAddress;

    @SerializedName("family")
    private List<FamilyMember> mFamily;

    public Employee(String fName, int age, String mail, Address address, List<FamilyMember>
mFamily) {
        this.mFirstName = fName;
        this.mAge = age;
        this.mMail = mail;
        this.mAddress = address;
        this.mFamily = mFamily;
    }
}
```

Modify the code as follows –

```
Gson gson = new Gson();

Address address = new Address("India", "New Delhi");

List<FamilyMember> family = new ArrayList<>();
family.add(new FamilyMember("wife", 30));
family.add(new FamilyMember("Daughter", 5));

Employee employee = new Employee("John", 30, "john@gmail.com", address, family);
String json = gson.toJson(employee);
```

This will convert employee object containing ArrayList into json –

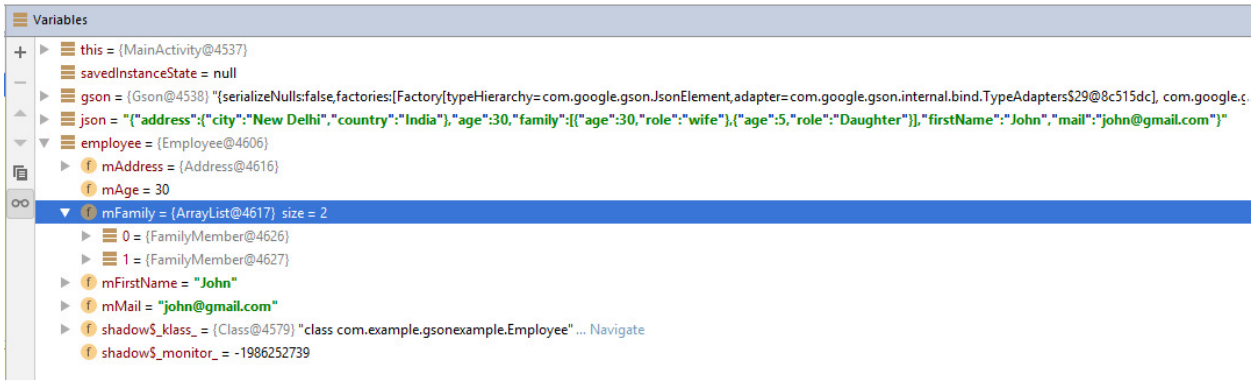
```
{
  "address": {
    "city": "New Delhi",
    "country": "India"
```

```
  },
  "age": 30,
  "family": [
    {
      "age": 30,
      "role": "wife"
    },
    {
      "age": 5,
      "role": "Daughter"
    }
  ],
  "firstName": "John",
  "mail": "john@gmail.com"
}
```

If convert this json into employee object –

```
Gson gson = new Gson();

String json = "{\"address\":{\"city\":\"New
Delhi\",\"country\":\"India\"},\"age\":30,\"family\":[{\"age\":30,\"role\":\"wife\"},{
\"age\":5,\"role\":\"Daughter\"}],\"firstName\":\"John\",\"mail\":\"john@gmail.com\"}"
;
Employee employee = gson.fromJson(json, Employee.class);
```

This also works –



If we directly parse the ArrayList<FamilyMember> to json –

```
Gson gson = new Gson();

List<FamilyMember> family = new ArrayList<>();
family.add(new FamilyMember("wife", 30));
family.add(new FamilyMember("Daughter", 5));

String json = gson.toJson(family);
```

this will be converted to json as array of objects –

[{"age":30,"role":"wife"},{"age":5,"role":"Daughter"}]

We can convert json to FamilyMember [ ] as –

```
Gson gson = new Gson();

String json = "[{\"age\":30,\"role\":\"wife\"},{\"age\":5,\"role\":\"Daughter\"}]";
FamilyMember[] family = gson.fromJson(json, FamilyMember[].class);
```

This converts the json into FamilyMember [ ] as shown below –

```
Variables
this = {MainActivity@4537}
savedInstanceState = null
gson = {Gson@4539} "{serializeNulls:false,factories:[Factory[typeHierarchy=com.google.gson.JsonElement,adapter=
json = "[{"age":30,"role":"wife"},{"age":5,"role":"Daughter"}]"
family = {FamilyMember[2]@4601}
    0 = {FamilyMember@4612}
        mAge = 30
        mRole = "wife"
        shadow$_klass_ = {Class@4579} "class com.example.gsonexample.FamilyMember" ... Navigate
        shadow$_monitor_ = -1966064258
    1 = {FamilyMember@4613}
        mAge = 5
        mRole = "Daughter"
        shadow$_klass_ = {Class@4579} "class com.example.gsonexample.FamilyMember" ... Navigate
        shadow$_monitor_ = -1965374753
```

If we want to convert it to ArrayList<> we have to use TypeToken -

```
Gson gson = new Gson();

String json = "[{\"age\":30,\"role\":\"wife\"},{\"age\":5,\"role\":\"Daughter\"}]";
Type familyType = new TypeToken<ArrayList<FamilyMember>>(){}.getType();
ArrayList<FamilyMember> family = gson.fromJson(json, familyType);
```

This is convert the json array into an ArrayList<FamilyMember>

```
Variables
this = {MainActivity@4540}
savedInstanceState = null
gson = {Gson@4542} "{serializeNulls:false,factories:[Factory[typeHierarchy=com.google.gson.JsonElement,adapter=com.google.
json = "[{"age":30,"role":"wife"},{"age":5,"role":"Daughter"}]"
familyType = {$Gson$Types$ParameterizedTypeImpl@4544} "java.util.ArrayList<com.example.gsonexample.FamilyMember>"
family = {ArrayList@4607} size = 2
    0 = {FamilyMember@4619}
        mAge = 30
        mRole = "wife"
        shadow$_klass_ = {Class@4537} "class com.example.gsonexample.FamilyMember" ... Navigate
        shadow$_monitor_ = -1916505999
    1 = {FamilyMember@4620}
        mAge = 5
        mRole = "Daughter"
        shadow$_klass_ = {Class@4537} "class com.example.gsonexample.FamilyMember" ... Navigate
        shadow$_monitor_ = -1890812330
```

Excluding Fields from Serialization and De-serialization –

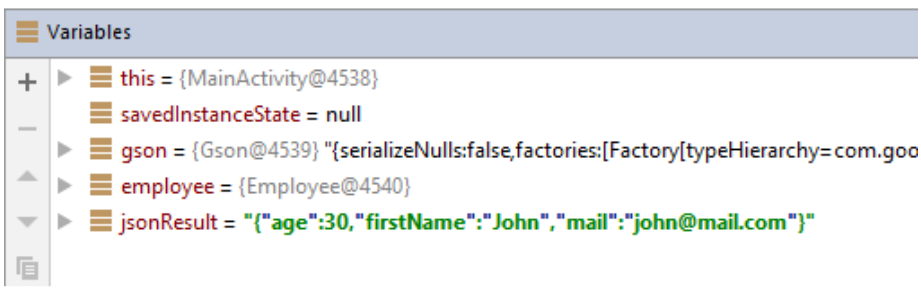
We can exclude some fields using java keyword 'transient'. Eg

```
public class Employee {
    private String firstName;
    private int age;
    private String mail;
    private transient String password;

    public Employee(String firstName, int age, String mail, String password) {
        this.firstName = firstName;
        this.age = age;
        this.mail = mail;
        this.password = password;
    }
}
```

Running the below code shows the same –

```
Gson gson = new Gson();

Employee employee = new Employee("John",30,"john@mail.com", "adfsa");
String jsonResult = gson.toJson(employee);
```

The password will be omitted while converting into json string as shown below –



And if we de-serialize the json into java object, the omitted field will not be converted into deserialized object, and the resultant object will contain null value in corresponding field.

The same functionality we can achive using @Expose annotation. eg

```
public class Employee {
    @Expose                          //(serialize = true, deserialize = true)
    private String firstName;

    @Expose(serialize = false)      // should not appear in json, default: true
    private int age;

    @Expose(deserialize = false)    // json value should not be copied to this field
    private String mail;            // default : serialize = true

    private String password;       //(serialize = false, deserialize = false)

    public Employee(String firstName, int age, String mail, String password) {
        this.firstName = firstName;
        this.age = age;
        this.mail = mail;
        this.password = password;
    }
}
```

But in this case we can't use normal Gson object, instead we have to use GsonBuilder.

```
 Gson gson = new GsonBuilder()
        .excludeFieldsWithoutExposeAnnotation()
        .create();

Employee employee = new Employee("John",30,"john@mail.com", "adfsa");
String jsonResult = gson.toJson(employee);
```

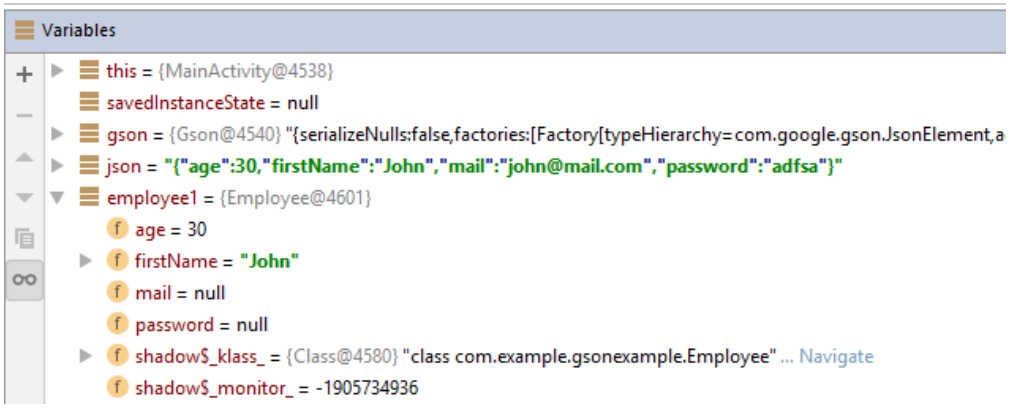
Running this code gives the following output –

{"firstName":"John","mail":"john@mail.com"}

Now, if we convert the json into employee object –

```
Gson gson = new GsonBuilder()
        .excludeFieldsWithoutExposeAnnotation()
        .create();

String json =
"{\"age\":30,\"firstName\":\"John\",\"mail\":\"john@mail.com\",\"password\":\"adfsa\"}
";
Employee employee1 = gson.fromJson(json, Employee.class);
```

it will translate the firstname and age only –



This is the way we can exclude certain fields in serialization and de-serialization process.